

TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN



PHÁT TRIỂN PHẦN MỀM MÃ NGUỒN MỞ

Xây Dựng Ứng Dụng

Nhận Diện Vật Thể dựa trên mô hình CNN

Thành viên nhóm:
Nguyễn Thanh Bình - 3121410082

Giáo viên hướng dẫn:
ThS. Từ Lăng Phiêu

TP. HỒ CHÍ MINH, THÁNG 5/2024

Mục lục

1	Giới Thiệu Chung	3
1.1	Python là gì?	3
1.2	Django là gì?	3
1.3	Tensorflow là gì?	4
1.4	Tensorflow.js là gì?	5
1.5	OpenCV-Python(cv2) là gì?	5
1.6	SQLite là gì?	6
2	Phân Tích Đề Tài	7
2.1	Lý do lựa chọn đề tài.	7
2.2	Mục tiêu	7
3	CNN (Convolutional Neural Networks) là gì?	8
3.1	Giới thiệu chung?	8
3.2	Cách Convolutional Neural Networks hoạt động?	8
3.3	MobileNet là gì?	10
3.4	Các Đặc điểm Chính của MobileNet	10
4	Xây Dựng Website	11
4.1	Cài đặt môi trường cần thiết.	11
4.1.1	Cài Đặt Python 3.7	11
4.1.2	Cài Đặt Virtual Environment và Các Gói Cần Thiết	11
4.1.3	Tạo và Kích Hoạt Môi Trường Ảo	11
4.1.4	Cài Đặt TensorFlow:2.4.1, numpy, opencv-python, Django và các thư viện cần thiết khác.	11
4.1.5	Tạo Django Project với django-admin	11
4.1.6	Khởi tạo Project mẫu với manage.py	11
4.1.7	Tạo Django App với manage.py	12
4.2	Phân tích và xây dựng Database(SQLite)	12
4.3	Tìm kiếm và phân tích mô hình Model-View-Template (MVT).	13
4.3.1	Mô hình MVT là gì?	13
4.3.2	Xây dựng các Model	13
4.3.3	Xây dựng View	15
4.3.4	Xây dựng Template	18
4.4	Xây dựng model CNN.	18
5	Các Chức Năng Đã Xây Dựng	23
5.1	Trang chủ.	23
5.2	Trang Dự Án Mới	23
5.2.1	Trang Huấn Luyện Tiêu Chuẩn	24
5.2.2	Xử Lý Nhật Ký Huấn Luyện	30
5.2.3	Hiển Thị Phân Tích Mất Mát và Độ Chính Xác của Hình Ảnh	31
5.2.4	Trang Huấn Luyện Tùy Chỉnh	34
5.2.5	Trang Nhập Mô Hình Huấn Luyện	38
5.3	Trang Quản Trị	40
5.3.1	Đăng Nhập	40



5.3.2	Bảng Điều Khiển Quản Trị	41
5.3.3	Xem Thông Tin Mô Hình Bộ Dữ Liệu	41
5.3.4	Tạo Mô Hình Bộ Dữ Liệu Mới	42
5.3.5	Xóa Mô Hình Bộ Dữ Liệu	42
5.3.6	Chỉnh Sửa Mô Hình Bộ Dữ Liệu	43
6	Nhiệm vụ, vai trò của từng thành viên trong nhóm.	44

1 Giới Thiệu Chung

1.1 Python là gì?

Python là một ngôn ngữ lập trình mạnh mẽ, dễ học và đa mục đích. Nó được tạo ra bởi Guido van Rossum và được phát hành lần đầu vào năm 1991. Python có cú pháp đơn giản và rõ ràng, giúp người dùng dễ dàng đọc, viết và hiểu code.



Hình 1: Biểu tượng của ngôn ngữ lập trình Python.

Với Python có thể phát triển ứng dụng web, ứng dụng di động, ứng dụng, trò chơi, công cụ phân tích dữ liệu, trí tuệ nhân tạo và nhiều hơn nữa. Nó có một cộng đồng lớn và phong phú, cung cấp nhiều thư viện và framework hữu ích để hỗ trợ phát triển.

Python cũng là một ngôn ngữ linh hoạt, có khả năng tích hợp và tương tác với các ngôn ngữ khác như C/C++ và Java. Nó có một cú pháp đơn giản và hỗ trợ nhiều phong cách lập trình, bao gồm lập trình hướng đối tượng, lập trình hàm và lập trình thủ tục. Tuy nhiên, Python thường chậm hơn so với các ngôn ngữ biên dịch và có thể gây ra các vấn đề về quản lý bộ nhớ tự động và không tương thích giữa các phiên bản khác nhau.

1.2 Django là gì?

Django là một framework phát triển ứng dụng web được viết bằng Python. Nó được phát triển để giúp các nhà phát triển xây dựng các ứng dụng web mạnh mẽ, linh hoạt và dễ bảo trì. Django tuân theo kiến trúc MTV (Model-Template-View), giúp tách biệt rõ ràng giữa dữ liệu, giao diện và logic xử lý.



Hình 2: Biểu tượng của Django.

Django cung cấp nhiều tính năng mạnh mẽ và hỗ trợ một quy trình phát triển ứng dụng web nhanh chóng. Một số tính năng quan trọng của Django bao gồm:

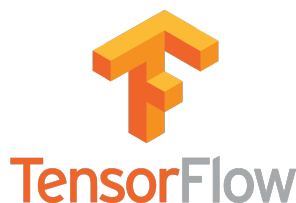
- ORM (Object-Relational Mapping): Django cung cấp một lớp trừu tượng để làm việc với cơ sở dữ liệu, cho phép sử dụng Python để tương tác với cơ sở dữ liệu thay vì viết các truy vấn SQL trực tiếp.
- Hỗ trợ mạnh mẽ cho URLs: Django có hệ thống URL rất linh hoạt và dễ sử dụng, giúp quản lý các đường dẫn của ứng dụng một cách dễ dàng.
- Giao diện quản trị tích hợp sẵn: Django đi kèm với một giao diện quản trị tích hợp sẵn, cho phép dễ dàng tạo, chỉnh sửa và xóa dữ liệu trong cơ sở dữ liệu mà không cần viết mã HTML hoặc JavaScript.
- Hệ thống mẫu (Template system): Django hỗ trợ hệ thống mẫu mạnh mẽ, cho phép tách biệt hoàn toàn giao diện người dùng với mã Python, giúp phát triển ứng dụng dễ dàng và linh hoạt hơn.
- Bảo mật: Django cung cấp nhiều tính năng bảo mật như xác thực, phân quyền và bảo vệ chống tấn công CSRF (Cross-Site Request Forgery).
- Cộng đồng phát triển mạnh mẽ: Django là một dự án mã nguồn mở được phát triển và duy trì bởi một cộng đồng lớn. Điều này có nghĩa là có thể tìm thấy nhiều tài liệu, thư viện bổ sung và hỗ trợ từ cộng đồng Django.

Django đã được sử dụng rộng rãi trong nhiều ứng dụng web lớn và nổi tiếng như Instagram, Pinterest,...

Mặc dù Django là một framework phát triển web mạnh mẽ với nhiều ưu điểm, nhưng cũng có một số nhược điểm. Đối với các ứng dụng lớn và phức tạp, việc mở rộng Django có thể trở nên khó khăn và gặp phải các vấn đề về hiệu suất. Ngoài ra, Django có một ngưỡng học đầu vào khá cao và có cấu trúc phức tạp, làm cho việc học và triển khai ứng dụng trở nên khó khăn đối với người mới bắt đầu. Đối với các ứng dụng nhỏ, Django có thể cảm thấy quá nặng và cồng kềnh.

1.3 Tensorflow là gì?

TensorFlow là một thư viện mã nguồn mở dành cho học máy và học sâu, được phát triển bởi Google Brain Team. Thư viện này cung cấp một API linh hoạt và mạnh mẽ cho Python, cùng với các hỗ trợ API cho các ngôn ngữ khác như C++ và Java, cho phép các nhà phát triển dễ dàng xây dựng và triển khai các mô hình học máy phức tạp.



Hình 3: biểu tượng của Tensorflow.

TensorFlow được sử dụng rộng rãi trong nhiều lĩnh vực như nhận diện giọng nói, xử lý ngôn ngữ tự nhiên, nhận diện hình ảnh, và nhiều ứng dụng trí tuệ nhân tạo khác. Nó cung cấp khả

năng tính toán trên cả CPU và GPU, điều này làm cho các mô hình học máy có thể được huấn luyện nhanh chóng hơn nhiều.

Một điểm mạnh chính của TensorFlow là kiến trúc dựa trên đồ thị tính toán, nơi các nút trong đồ thị đại diện cho các hoạt động toán học, và các cạnh đại diện cho các mảng đa chiều (tensors) mà dữ liệu được truyền qua. Điều này cho phép tối ưu hóa hiệu suất và là một yếu tố quan trọng cho phép triển khai mô hình một cách hiệu quả trên cả máy tính cá nhân và máy chủ. Tuy nhiên, TensorFlow cũng có nhược điểm như cú pháp phức tạp và khó hiểu đối với người mới bắt đầu, và đôi khi có thể gặp phải các vấn đề về hiệu suất và tốc độ khi xử lý dữ liệu lớn.

1.4 Tensorflow.js là gì?

TensorFlow.js là một thư viện của TensorFlow được thiết kế để phát triển và huấn luyện mô hình học máy ngay trong trình duyệt hoặc trên nền tảng Node.js. Nó cho phép các nhà phát triển sử dụng JavaScript để xây dựng, huấn luyện, và triển khai các mô hình học máy trực tiếp trên môi trường web, mà không cần đến các ngôn ngữ lập trình máy chủ phức tạp hơn như Python.



Hình 4: biểu tượng của Django.

Một điểm mạnh lớn của TensorFlow.js là khả năng tích hợp trực tiếp vào các ứng dụng web, làm cho các ứng dụng AI trở nên dễ tiếp cận hơn cho người dùng cuối mà không cần cài đặt thêm phần mềm. Nó cũng hỗ trợ tính toán trên GPU thông qua WebGL, giúp tăng tốc độ xử lý và hiệu quả của mô hình.

TensorFlow.js có khả năng chạy mô hình học máy trực tiếp trong trình duyệt do đó, tích hợp dễ dàng với các framework web như React và Vue.js, và có một cộng đồng lớn và tích cực. Tuy nhiên, TensorFlow.js có thể gặp phải các vấn đề về hiệu suất và tốc độ khi xử lý các tác vụ phức tạp trên dữ liệu lớn trong môi trường trình duyệt.

1.5 OpenCV-Python(cv2) là gì?

OpenCV-Python là một thư viện Python được sử dụng rộng rãi trong xử lý hình ảnh và thị giác máy tính. Nó là một giao diện Python của OpenCV (Open Source Computer Vision Library) – một thư viện lớn và mạnh mẽ bao gồm hàng nghìn thuật toán từ cơ bản đến nâng cao trong các lĩnh vực như nhận diện khuôn mặt, theo dõi chuyển động, và nhiều hơn nữa.

Thư viện này cung cấp công cụ để thực hiện các tác vụ như đọc và ghi hình ảnh, xử lý hình ảnh (như lọc, biến đổi, và phát hiện cạnh), và giải thuật nhận diện đối tượng. OpenCV-Python được sử dụng phổ biến trong các ứng dụng từ an ninh, điều khiển robot đến các ứng dụng nội dung số, nơi cần phân tích và xử lý hình ảnh một cách tự động và hiệu quả.

OpenCV-Python có tính linh hoạt, tích hợp dễ dàng với Python và các thư viện khác như NumPy, và có một cộng đồng lớn và tích cực. Nó cũng hỗ trợ nhiều hệ thống máy tính khác nhau và hệ điều hành. Tuy nhiên, một số nhược điểm của OpenCV-Python có thể bao gồm tài liệu không đồng nhất và cú pháp không rõ ràng đối với người mới bắt đầu.



Hình 5: biểu tượng của cv2.

1.6 SQLite là gì?

SQLite là một hệ quản trị cơ sở dữ liệu quan hệ nhẹ, được lưu trữ dưới dạng một tệp duy nhất trên đĩa. Nó rất nhỏ gọn và không yêu cầu một máy chủ riêng biệt hoặc cấu hình phức tạp, điều này làm cho nó trở thành lựa chọn phổ biến cho các ứng dụng di động, ứng dụng máy tính để bàn, và các thiết bị nhúng. SQLite cung cấp một phương tiện đơn giản và hiệu quả để lưu trữ dữ liệu mà không cần đến sự phức tạp của các hệ thống quản lý cơ sở dữ liệu lớn hơn.



Hình 6: biểu tượng của Django.

SQLite hỗ trợ gần như toàn bộ ngôn ngữ truy vấn SQL và rất nhiều tính năng của các hệ quản trị cơ sở dữ liệu lớn hơn, nhưng với một mức độ tinh giản để giữ cho nó nhẹ và dễ sử dụng. Mặc dù nó không phù hợp cho các ứng dụng cần xử lý một khối lượng lớn dữ liệu hoặc hỗ trợ nhiều người dùng đồng thời, SQLite lại hoàn hảo cho việc phát triển nhanh chóng, thử nghiệm, và sử dụng trong các ứng dụng với yêu cầu quản lý dữ liệu không quá phức tạp.

SQLite không yêu cầu cài đặt hoặc quản lý máy chủ cơ sở dữ liệu riêng biệt, và hiệu suất tốt cho các ứng dụng nhỏ và trung bình. SQLite cũng hỗ trợ nhiều ngôn ngữ lập trình thông qua các giao diện API, bao gồm Python, Java, và C++. Tuy nhiên, do thiết kế nhẹ nhàng, SQLite có thể không phù hợp cho các ứng dụng có yêu cầu hiệu suất cao hoặc có quy mô lớn.



2 Phân Tích Đề Tài

2.1 Lý do lựa chọn đề tài.

Xu hướng thị giác máy tính và phát triển website hiện nay ngày càng phát triển mạnh mẽ với nhu cầu thị trường đầy tiềm năng. Còn là một bước đi chiến lược để khai thác hai trong số những lĩnh vực công nghệ phát triển nhanh chóng nhất hiện nay. Đề tài này mang đến cơ hội khám phá cách thức công nghệ học sâu, đặc biệt là Mạng Nơ-ron Tích Chập (CNN), có thể được áp dụng để giải quyết một loạt các vấn đề thực tế từ nhận diện và phân loại vật thể trong hình ảnh. Đây là một công nghệ then chốt trong nhiều ứng dụng quan trọng, bao gồm các hệ thống giám sát an ninh, hệ thống đề xuất sản phẩm trong thương mại điện tử, và các giải pháp tự động hóa trong công nghiệp sản xuất.

Ngoài ra, việc lựa chọn Python, một ngôn ngữ lập trình với một cộng đồng sử dụng rộng rãi và nhiều thư viện phát triển mạnh, giúp tối đa hóa tính tiếp cận và hiệu quả phát triển của dự án. Python cung cấp các công cụ mạnh mẽ như TensorFlow và Keras, làm cho việc triển khai các mô hình CNN trở nên trực quan và dễ dàng hơn bao giờ hết. Thêm vào đó, triển khai dự án trên nền tảng web không chỉ giúp đơn giản hóa việc truy cập và sử dụng công nghệ nhận diện hình ảnh cho người dùng cuối mà còn mở rộng phạm vi ảnh hưởng và ứng dụng của nó trong thực tiễn.

Trong quá trình tìm hiểu chúng em thấy rất hứng thú với khả năng xây dựng website kết hợp với CNN. Chúng em đặc biệt chú ý tới Django – Web Framework, một ứng dụng chuyên dành để xây dựng các website bằng ngôn ngữ lập trình Python, nên chúng em quyết định sử dụng Django để xây dựng một trang website dự đoán vật thể bằng những hình ảnh mà người dùng cung cấp hoặc có sẵn.

2.2 Mục tiêu

- Phát Triển Mô Hình CNN:** Thiết kế và huấn luyện một mô hình nhận diện vật thể hiệu quả dựa trên mạng nơ-ron tích chập, sử dụng Python và các thư viện hỗ trợ như TensorFlow. Mục tiêu là để mô hình có thể nhận diện và phân loại vật thể trong hình ảnh với độ chính xác cao nhất.
- Xây Dựng Website:** Phát triển một trang web đơn giản để hiển thị và điều khiển mô hình CNN. Website này sẽ cho phép người dùng tải lên hình ảnh và nhận kết quả nhận diện vật thể. Giao diện người dùng sẽ được thiết kế để đảm bảo dễ sử dụng và trực quan.
- Tích Hợp Mô Hình Vào Website:** Tích hợp mô hình CNN vào backend của website để xử lý các yêu cầu từ người dùng và trả về kết quả nhận diện vật thể.
- Tối Ưu Hóa:** Thực hiện đảm bảo mô hình và website hoạt động ổn định và hiệu quả. Tối ưu hóa hiệu năng và độ chính xác của mô hình, cũng như cải thiện trải nghiệm người dùng trên website.

3 CNN (Convolutional Neural Networks) là gì?

3.1 Giới thiệu chung?

CNN, viết tắt của Convolutional Neural Networks, là một lớp mạng nơ-ron nhân tạo chuyên biệt được thiết kế để xử lý và phân tích dữ liệu có cấu trúc lưới, chẳng hạn như hình ảnh. Mạng này bắt nguồn từ cách thức não bộ sinh vật nhìn và nhận biết thế giới xung quanh thông qua thị giác, giúp máy tính hiểu và tương tác với thế giới hình ảnh một cách hiệu quả.

Cấu trúc của CNN bao gồm nhiều lớp, trong đó lớp tích chập (convolutional layer) là trung tâm, nơi các bộ lọc (filters) được áp dụng lên dữ liệu đầu vào để trích xuất đặc trưng. Điều này giúp CNN có khả năng phát hiện các cạnh, góc cạnh, và các đặc trưng phức tạp khác trong hình ảnh. Lớp tổng hợp (pooling layer) giảm kích thước không gian của đặc trưng, giảm thiểu số lượng tham số và tính toán, giúp mô hình trở nên hiệu quả hơn. Cuối cùng, các lớp kết nối đầy đủ (fully connected layers) tổng hợp các đặc trưng đã được trích xuất để thực hiện phân loại hoặc nhận dạng.

Khả năng tự động học và trích xuất đặc trưng phức tạp từ dữ liệu hình ảnh, mà không cần sự can thiệp hoặc lập trình đặc trưng cụ thể từ phía con người, làm cho CNN trở thành công cụ vô cùng mạnh mẽ trong lĩnh vực thị giác máy tính. Từ nhận dạng khuôn mặt, phân loại ảnh, đến phát hiện vật thể và phân tích video, CNN đã mở ra một loạt các ứng dụng thực tiễn, thúc đẩy tiến bộ nhanh chóng trong cả nghiên cứu và ứng dụng thương mại của trí tuệ nhân tạo và học máy.

3.2 Cách Convolutional Neural Networks hoạt động?

1. Lớp Tích chập (Convolutional Layer): là phần xây dựng cốt lõi của Mạng tích chập thực hiện hầu hết các công việc tính toán nặng nhọc. Các lớp convolution được sử dụng để áp dụng các bộ lọc (hay kernels) lên ảnh đầu vào để phát hiện, trích xuất các đặc trưng từ dữ liệu đầu vào, thường là ảnh, một cách tự động và hiệu quả..

Khi chúng ta có một tập dataset chứa các hình ảnh đã được gán nhãn, chúng ta sẽ chuyển đổi mỗi hình ảnh này thành một ma trận số tương ứng. Sau đó, chúng ta sẽ tạo ra các kernel, có thể là các ma trận nhỏ với kích thước 3×3 , và áp dụng chúng trên toàn bộ bức ảnh.

Mỗi lần áp dụng một kernel, chúng ta sẽ nhận được một ma trận kết quả (output), và với nhiều kernel, chúng ta sẽ có nhiều ma trận kết quả tương ứng. Tất cả các ma trận kết quả này sẽ được kết hợp lại thành một tensor ba chiều có chiều sâu bằng số lượng kernel đã sử dụng. Điều này giúp mạng CNN học được một đa dạng các đặc trưng và biểu diễn của ảnh từ các lớp convolution.

2. Lớp Ghép (Pooling Layer): Các lớp ghép giảm kích thước không gian của đầu vào, giảm độ phức tạp tính toán và số lượng tham số trong mạng. Giúp giảm lượng tính toán cần thiết và nguy cơ overfitting.

Có hai loại phổ biến của Pooling Layer: Max Pooling và Average Pooling. Trong Max Pooling, chúng ta chọn giá trị lớn nhất từ mỗi vùng pooling, trong khi đó, trong Average Pooling, chúng ta tính trung bình các giá trị trong vùng pooling. Cả hai loại đều giữ lại các đặc điểm quan trọng của ảnh và giảm nhiễu trong dữ liệu.

Quá trình pooling được thực hiện bằng cách chia feature map thành các vùng không gian và chọn giá trị đại diện cho mỗi vùng. Kích thước và bước nhảy của vùng pooling có thể được xác định trước, thường là (2×2) với stride là 2.

Pooling Layer thường được sử dụng sau mỗi lớp convolutional trong CNN để giảm kích thước của feature map trước khi đi vào các lớp fully connected. Điều này giúp tăng tốc độ huấn luyện và giảm lượng thông tin không cần thiết.

3. Lớp Kích hoạt (Activation Layer): là một thành phần quan trọng, đặc biệt là trong các mạng Convolutional Neural Network (CNN). Nó giúp mạng neural học được các biểu diễn phi tuyến tính và tạo ra đầu ra không tuyến tính, làm cho mô hình có khả năng học được các mối quan hệ phức tạp giữa các đặc trưng đầu vào và đầu ra.

Công dụng chính của Activation Layer là áp dụng một hàm kích hoạt (activation function) cho mỗi đầu ra của các nơ-ron trong mạng neural. Hàm kích hoạt này thường là một hàm phi tuyến tính và thường được sử dụng là ReLU (Rectified Linear Unit), Tanh (Tangential Hyperbolic), hoặc Sigmoid.

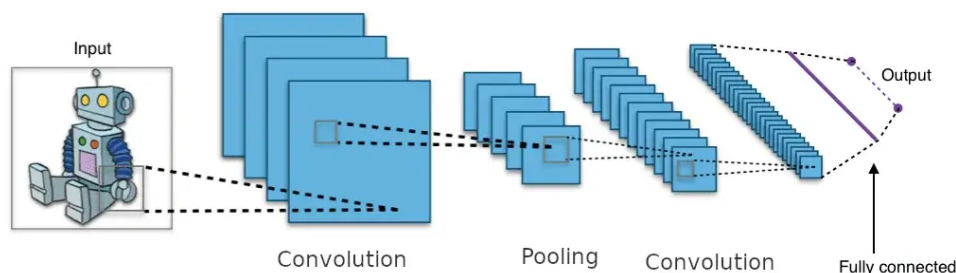
ReLU (Rectified Linear Unit): Hàm này trả về giá trị là 0 nếu đầu vào là âm, và giữ nguyên giá trị nếu đầu vào là dương. Điều này giúp loại bỏ hiện tượng vanishing gradient và tăng tốc độ học của mạng. Tanh (Tangential Hyperbolic): Hàm này ánh xạ các giá trị đầu vào sang một phạm vi từ -1 đến 1. Nó thường được sử dụng trong các lớp ẩn của mạng neural. Sigmoid: Hàm này ánh xạ các giá trị đầu vào sang một phạm vi từ 0 đến 1. Nó thường được sử dụng để ánh xạ đầu ra của mạng neural thành xác suất trong các bài toán phân loại nhị phân.

4. Lớp Kết nối Đầy đủ (Fully Connected Layer): là lớp cuối cùng trong một mạng neural truyền thẳng, trong đó mỗi nơ-ron kết nối với tất cả các nơ-ron trong lớp trước đó.

Chức năng chính của Fully Connected Layer là học các biểu diễn phức tạp từ dữ liệu đầu vào và tạo ra đầu ra dự đoán. Mỗi kết nối giữa các nơ-ron được xác định bởi một trọng số riêng, và mỗi nơ-ron có một hệ số bias riêng. Đầu ra của mỗi nơ-ron thường được truyền qua một hàm kích hoạt phi tuyến tính như ReLU, Tanh hoặc Sigmoid để tạo ra đầu ra không tuyến tính.

Fully Connected Layer thường được sử dụng ở cuối mạng neural để thực hiện các tác vụ như phân loại, dự đoán và regression. Trong các mạng CNN, Fully Connected Layer thường được sử dụng sau các lớp Convolutional và Pooling để thực hiện phân loại đối tượng trong hình ảnh.

Để giảm overfitting, Fully Connected Layer thường được kết hợp với các lớp Dropout, một kỹ thuật thảo luận giảm ngẫu nhiên một số lượng nơ-ron trong quá trình huấn luyện. Điều này giúp mô hình tránh việc phụ thuộc quá mức vào một số đặc trưng cụ thể.

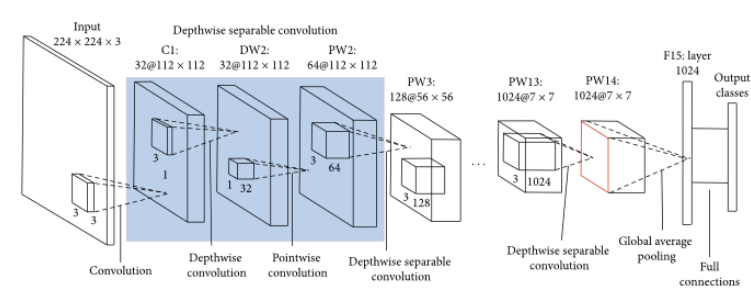


3.3 MobileNet là gì?

MobileNet là kiến trúc mạng nơ-ron tích chập được thiết kế đặc biệt cho các thiết bị di động với tài nguyên tính toán hạn chế. Được phát triển bởi các nhà nghiên cứu tại Google, nó lần đầu tiên được giới thiệu trong bài báo "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications" vào năm 2017. Mục tiêu chính của MobileNet là cung cấp một mô hình hiệu quả về mặt tính toán và dung lượng bộ nhớ mà vẫn giữ được độ chính xác cao, phù hợp cho các ứng dụng thị giác máy tính trên thiết bị di động.

3.4 Các Đặc điểm Chính của MobileNet

- **Tích chập Phân tách theo Chiều sâu (Depthwise Separable Convolution):** Đây là đặc điểm nổi bật nhất của MobileNet. Nó sử dụng tích chập phân tách theo chiều sâu bao gồm:
 1. **Tích chập theo chiều sâu (Depthwise Convolution):** Áp dụng một bộ lọc cho từng kênh đầu vào riêng biệt, giảm đáng kể số lượng phép tính.
 2. **Tích chập theo điểm (Pointwise Convolution):** Sử dụng tích chập 1×1 để kết hợp các kênh đầu ra từ tích chập theo chiều sâu, đạt được độ sâu mong muốn ở đầu ra.
- **Gọn nhẹ và Hiệu quả:** Nhờ vào tích chập phân tách theo chiều sâu, MobileNet đòi hỏi ít tham số hơn và giảm đáng kể gánh nặng tính toán so với các kiến trúc CNN truyền thống, làm cho nó trở thành lý tưởng cho các thiết bị di động và hệ thống nhúng.
- **Khả năng Tùy Chỉnh:** MobileNet cung cấp hai siêu tham số có thể điều chỉnh để cân bằng giữa độ chính xác và hiệu suất:
 1. **Nhân tử Chiều rộng (Width Multiplier):** Cho phép mô hình được làm mỏng đồng đều trên toàn bộ mạng. Nó thu nhỏ số kênh trong mạng theo một tỷ lệ nhất định, giảm số lượng tham số và phép tính.
 2. **Nhân tử Độ phân giải (Resolution Multiplier):** Điều chỉnh độ phân giải của đầu vào, giảm tải tính toán bằng cách giảm độ phân giải của hình ảnh đầu vào.
- **Ứng dụng Rộng rãi:** MobileNet không chỉ giới hạn ở các tác vụ nhận dạng hình ảnh; nó còn có thể được sử dụng trong nhiều ứng dụng khác như ước lượng tư thế, nhận diện khuôn mặt, v.v...).





4 Xây Dựng Website

4.1 Cài đặt môi trường cần thiết.

Do một số package không còn hỗ trợ trên Windows và để tránh lỗi không mong muốn trong quá trình cài đặt cấu hình tối thiểu là:

1. Hệ điều hành: Linux
2. Python Version: ≥ 3.7
3. Tensorflow Version: ≥ 2.4

4.1.1 Cài Đặt Python 3.7

```
sudo apt update  
sudo apt install python3.7
```

4.1.2 Cài Đặt Virtual Environment và Các Gói Cần Thiết

```
sudo apt install python3-pip  
sudo pip3 install virtualenv
```

4.1.3 Tạo và Kích Hoạt Môi Trường Ảo

```
virtualenv -p /usr/bin/python3.7 env  
source env/bin/activate
```

4.1.4 Cài Đặt TensorFlow:2.4.1, numpy, opencv-python, Django và các thư viện cần thiết khác.

```
pip install tensorflow  
pip install django
```

4.1.5 Tạo Django Project với django-admin

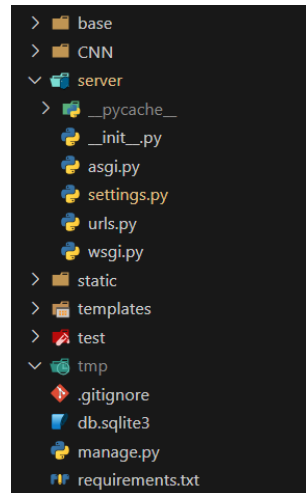
```
django-admin startproject backend
```

4.1.6 Khởi tạo Project mẫu với manage.py

```
cd backend  
python manage.py startproject server
```

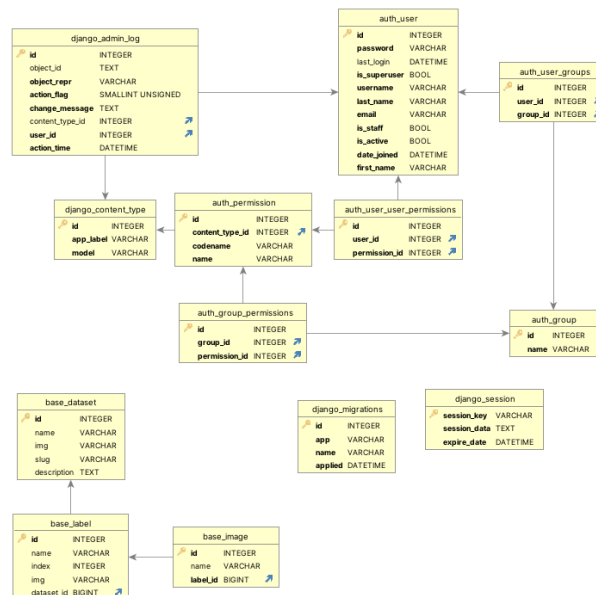
4.1.7 Tạo Django App với manage.py

```
python manage.py startapp base
```



Hình 7: Cấu trúc thư mục.

4.2 Phân tích và xây dựng Database(SQLite)



Hình 8: Sơ đồ ERD database.

4.3 Tìm kiếm và phân tích mô hình Model-View-Template (MVT).

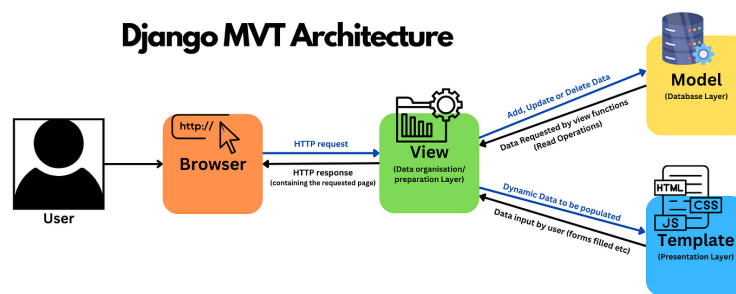
4.3.1 Mô hình MVT là gì?

Mô hình Model-View-Template (MVT) là một kiến trúc phổ biến được sử dụng trong framework phát triển web Django. Mô hình này tương tự như mô hình Model-View-Controller (MVC), nhưng thay vì Controller, Django sử dụng Template.

Model: Mô hình đại diện cho dữ liệu của ứng dụng và quản lý truy cập và thao tác dữ liệu. Mỗi mô hình trong Django tương ứng với một bảng trong cơ sở dữ liệu, và các truy vấn được thực hiện thông qua các đối tượng mô hình.

View: Xem là các thành phần xử lý logic và tương tác với người dùng. Trong Django, các xem thường được triển khai dưới dạng các hàm hoặc lớp, và chịu trách nhiệm xử lý yêu cầu từ người dùng và tạo ra các phản hồi tương ứng.

Template: Mẫu là các tập tin chứa mã HTML được sử dụng để hiển thị dữ liệu cho người dùng. Django sử dụng mẫu để tạo ra giao diện người dùng cuối cùng dựa trên dữ liệu được cung cấp từ các xem.



Hình 9: Mô hình MVT trong django.

4.3.2 Xây dựng các Model

4.3.2. Model Dataset

```
class Dataset(models.Model):
    name = models.CharField(null=True, max_length=100)
    slug = models.SlugField(null=True)
    description = models.TextField(null=True)
    img = models.CharField(null=True, max_length=200)

    def __str__(self):
        return f"{self.name}"
```

- **name:** Đây là một trường kiểu CharField trong Django, được sử dụng để lưu trữ tên của dataset. Trường này được đặt là null=True, cho phép giá trị của trường này có thể là null (trống) trong cơ sở dữ liệu. max_length=100 xác định độ dài tối đa của chuỗi ký tự cho trường này là 100.

- **slug:** Đây là một trường kiểu `SlugField`, được sử dụng để lưu trữ một phiên bản chuẩn hóa của tên dataset, thường được sử dụng trong URL. Trường này cũng được đặt là `null=True`, cho phép giá trị của trường này có thể là null.
- **description:** Đây là một trường kiểu `TextField`, được sử dụng để lưu trữ mô tả của dataset. Trường này cũng được đặt là `null=True`, cho phép giá trị của trường này có thể là null.
- **img:** Đây là một trường kiểu `CharField`, được sử dụng để lưu trữ URL hoặc đường dẫn tới hình ảnh đại diện cho dataset. `max_length=200` xác định độ dài tối đa của chuỗi ký tự cho trường này là 200. Trường này cũng được đặt là `null=True`, cho phép giá trị của trường này có thể là null.

4.3.2. Model Label

```
class Label(models.Model):
    name = models.CharField(null=True, max_length=100)
    index = models.IntegerField(null=True)
    dataset = models.ForeignKey(Dataset, on_delete=models.DO_NOTHING, null=True,
                               blank=True, default=None)

    def __str__(self):
        return f"{self.name}"
```

- **name:** Đây là một trường kiểu `CharField` trong Django, được sử dụng để lưu trữ tên của label. Trường này được đặt là `null=True`, cho phép giá trị của trường này có thể là null trong cơ sở dữ liệu. `max_length=100` xác định độ dài tối đa của chuỗi ký tự cho trường này là 100.
- **index:** Đây là một trường kiểu `IntegerField`, được sử dụng để lưu trữ chỉ số của label. Trường này cũng được đặt là `null=True`.
- **dataset:** Đây là một trường kiểu `ForeignKey`, được sử dụng để thiết lập mối quan hệ nhiều một với model `Dataset`. Trong trường hợp này, khi một dataset bị xóa, các label liên quan sẽ không bị ảnh hưởng (`on_delete=models.DO_NOTHING`). Trường này cũng được đặt là `null=True` và `blank=True` để cho phép giá trị của trường này có thể là null và để cho phép trường này có thể bỏ trống khi tạo mới một đối tượng `Label`.

4.3.2. Model Image

```
class Image(models.Model):
    name = models.CharField(max_length=100, null=True)
    label = models.ForeignKey(Label, on_delete=models.DO_NOTHING)

    def __str__(self):
        return f"{self.name}"
```

- **name:** Đây là một trường kiểu `CharField` trong Django, được sử dụng để lưu trữ tên của hình ảnh. `max_length=100` xác định độ dài tối đa của chuỗi ký tự cho trường này là 100. Trường này có thể là null (`null=True`).

- **label:** Đây là một trường kiểu `ForeignKey`, được sử dụng để thiết lập mối quan hệ nhiều một với model `Label`. Trong trường hợp này, khi một label bị xóa, các hình ảnh liên quan sẽ không bị ảnh hưởng (`on_delete=models.DO_NOTHING`).

4.3.3 Xây dựng View

1. Cấu hình routing

```
from django.urls import path, include
from . import views

urlpatterns = [
    path('', views.getting_started, name='getting_started'),
    path('train', views.train, name='train'),
    path('train/standard-training', views.standardTraining,
         name='standardTraining'),
    path('train/standard-training/<slug:slug>', views.standardTrainingDetail,
         name='standardTrainingDetail'),
    path('train/custom-training', views.customTraining, name='customTraining'),
    path('train-model', views.train_model, name='train-model'),
    path('migrate-data', views.migrate_data, name='migrate-data')
]
```

- `''`: Đường dẫn gốc, dẫn đến hàm `getting_started` trong module `views.py` khi truy cập vào trang chính của ứng dụng.
- `'train'`: Đường dẫn `/train`, dẫn đến hàm `train` trong module `views.py`.
- `'train/standard-training'`: Đường dẫn `/train/standard-training`, dẫn đến hàm `standardTraining` trong module `views.py`.
- `'train/standard-training/<slug:slug>'`: Đường dẫn `/train/standard-training/<slug:slug>`, với `'<slug:slug>'` là một phần của URL được truyền vào hàm `standardTrainingDetail` trong module `views.py`.
- `'train/custom-training'`: Đường dẫn `/train/custom-training`, dẫn đến hàm `customTraining` trong module `views.py`.
- `'train-model'`: Đường dẫn `/train-model`, dẫn đến hàm `train_model` trong module `views.py`.
- `'migrate-data'`: Đường dẫn `/migrate-data`, dẫn đến hàm `migrate_data` trong module `views.py`.

2. Cấu hình view

(a) Hàm `getting_started`

Listing 1: Định nghĩa hàm `getting_started`

```
def getting_started(request):
    return render(request, 'getting_started.html')
```

Hàm `getting_started` trả về trang "Getting Started" của ứng dụng web.

(b) Hàm `train`

Listing 2: Định nghĩa hàm train

```
def train(request):  
    return render(request, 'train.html')
```

(c) Hàm standardTrainingDetail

Listing 3: Định nghĩa hàm standardTrainingDetail

```
def standardTrainingDetail(request, slug):  
    sizes = int(request.GET.get('sizes', 10))  
    dataset = models.Dataset.objects.filter(slug=slug).first()  
    labels = models.Label.objects.filter(dataset=dataset)  
    images_by_labels = []  
    for label in labels:  
        images = models.Image.objects.filter(label=label)[:sizes]  
        images_by_labels.append((label, images))  
    labels_name = list([label.name for label in labels])  
    return render(request, 'standard_training.html', {'images_by_labels':  
        images_by_labels, 'dataset': dataset, 'labels': labels_name})
```

Hàm `standardTrainingDetail` lấy dữ liệu từ cơ sở dữ liệu Django để hiển thị chi tiết của một tập dữ liệu huấn luyện.

(d) Hàm standardTraining

Listing 4: Định nghĩa hàm standardTraining

```
def standardTraining(request):  
    datasets = models.Dataset.objects.all()  
    return render(request, 'standard_training_datasets.html', {'datasets':  
        datasets})
```

Hàm `standardTraining` lấy tất cả các tập dữ liệu từ cơ sở dữ liệu Django để hiển thị danh sách các tập dữ liệu huấn luyện chuẩn.

(e) Hàm customTraining

Listing 5: Định nghĩa hàm customTraining

```
def customTraining(request):  
  
    return render(request, 'custom-training.html')
```

Hàm `customTraining` trả về trang "Custom Training" của ứng dụng web, hiển thị danh sách các nhân có thể được huấn luyện tùy chỉnh.

(f) Hàm train_model

Listing 6: Định nghĩa hàm train_model

```
def train_model(request):  
    if request.method == 'POST':  
        body = json.loads(request.body)  
        data = body['trainingData']  
        setting = body['setting']
```

```
epochs=int(setting.get('epochs', 20))
patch_sizes=int(setting.get('patch_sizes', 64))
test_size=float(setting.get('test_size', 0.2))
optimizer=setting.get('optimizer', 'adam')

new_folder_name = str(uuid.uuid4())
new_folder_path = os.path.join(PROJECT_PATH, 'static', 'uploads',
                                new_folder_name)
os.makedirs(new_folder_path)

for item in data:
    label = item['label']
    images = item['images']

    label_folder_path = os.path.join(new_folder_path, label)
    os.makedirs(label_folder_path)

    index = 0
    for image_url in images:
        if image_url.startswith('data:image/'):
            image_data = image_url.split(',')[1]
            image_binary = base64.b64decode(image_data)
            new_image_name = f"base64_{index}.jpg"
            index += 1
            image_path_destination = os.path.join(label_folder_path,
                                                    new_image_name)
            with open(image_path_destination, 'wb') as file:
                file.write(image_binary)
        else:
            new_image_name = os.path.basename(image_url)
            image_path_source = PROJECT_PATH + image_url
            image_path_destination = os.path.join(label_folder_path,
                                                    new_image_name)
            shutil.copy(image_path_source, image_path_destination)

    model = Training(size=112, epochs=epochs,
                     patch_sizes=patch_sizes, test_size=test_size,
                     optimizer=optimizer, num_classes=num_classes, folder_path =
                     new_folder_path)
    model.build_model()
    train_generator, validation_generator = model.make_data()
    model.train_model(train_generator, validation_generator)
    model.save_model()

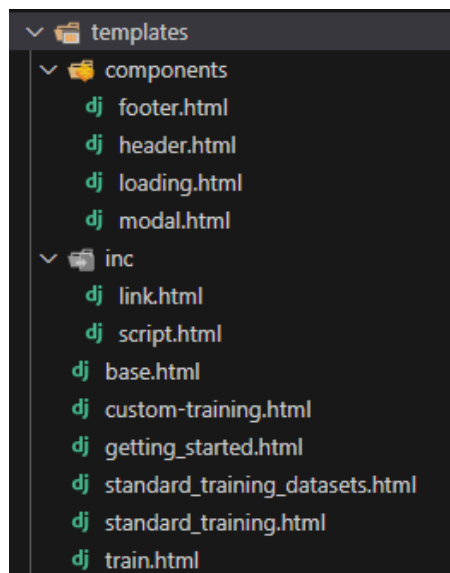
    for folder in class_names:
        subfolder_path = os.path.join(new_folder_path, folder)
        if os.path.isdir(subfolder_path):
            for file_name in os.listdir(subfolder_path):
                file_path = os.path.join(subfolder_path, file_name)
                try:
                    if os.path.isfile(file_path):
                        os.unlink(file_path)
```

```
except Exception as e:
    print(f"Can't delete file name is {file\_path}: {e}")
os.rmdir(subfolder_path)

return JsonResponse({'status': 'success', 'message': 'Model trained
successfully', 'uuid': new_folder_name})
else:
    return JsonResponse({'status': 'error', 'message': 'Invalid request
method'})
```

Hàm `train_model` xử lý yêu cầu huấn luyện mô hình từ dữ liệu được gửi lên từ giao diện người dùng.

4.3.4 Xây dựng Template



Trong folder template chứa mã các tập tin HTML. Templates cho phép bạn tách biệt giao diện người dùng từ mã logic của ứng dụng, giúp dễ dàng quản lý và tái sử dụng mã.

Trong folder inc có file link và script được dùng để thêm thư viện vào ví dụ như là bootstrap, jquery,...

Trong folder components được dùng để tái sử dụng lại các html bị trùng lặp ví dụ như header, modal, loading,...

4.4 Xây dựng model CNN.

Listing 7: Định nghĩa class Training

```
import cv2
from keras.applications.mobilenet import MobileNet
from keras.layers import GlobalAveragePooling2D, Dense, Dropout
from keras.models import Model
import tensorflowjs as tfjs
```

```
from keras.preprocessing.image import ImageDataGenerator
import keras
from keras.callbacks import ModelCheckpoint
import os
class Training:
    def __init__(self, size=112, epochs=20, patch_sizes=64, optimizer='adam',
        loss='categorical_crossentropy', test_size=0.2, num_classes=0, folder_path=''):
        self.model = None
        self.size = size
        self.epochs = epochs
        self.patch_sizes = patch_sizes
        self.optimizer = optimizer
        self.loss = loss
        self.num_classes = num_classes
        self.test_size = test_size
        self.folder_path = folder_path

    def build_model(self):
        base_model = MobileNet(include_top=False, weights="imagenet",
            input_shape=(self.size,self.size,3))
        x = base_model.output
        x = GlobalAveragePooling2D()(x)
        x = Dense(1024, activation='relu')(x)
        x = Dropout(0.25)(x)
        x = Dense(256, activation='relu')(x)
        x = Dropout(0.2)(x)
        outs = Dense(self.num_classes, activation='softmax')(x)

        for layer in base_model.layers:
            layer.trainable = False

        self.model = Model(inputs=base_model.inputs, outputs=outs)

    def make_data(self):
        train_datagen = ImageDataGenerator(preprocessing_function=
            keras.applications.mobilenet.preprocess_input,rotation_range=0.2,
                width_shift_range=0.2,
                height_shift_range=0.2,shear_range=0.3,zoom_range=0.5,
                horizontal_flip=True, vertical_flip=True,
                validation_split=self.test_size)

        train_generator = train_datagen.flow_from_directory(self.folder_path,
            target_size=(self.size,
                self.size),
            batch_size=self.patch_sizes,
            class_mode='categorical',
            subset='training')

        validation_generator = train_datagen.flow_from_directory(
            self.folder_path, # same directory as training data
            target_size=(self.size, self.size),
            batch_size=self.patch_sizes,
            class_mode='categorical',
```

```
subset='validation')
return train_generator, validation_generator

def train_model(self, train_generator, validation_generator):
    self.model.compile(optimizer=self.optimizer, loss=self.loss,
                        metrics=['accuracy'])
    self.model_checkpoint_path = os.path.join(self.folder_path, 'best.hdf5')
    checkpoint = ModelCheckpoint(self.model_checkpoint_path, monitor='val_loss',
                                save_best_only=True, mode='auto')
    callback_list = [checkpoint]

    self.model.fit(train_generator, steps_per_epoch=len(train_generator),
                  validation_data=validation_generator,
                  validation_steps=len(validation_generator),
                  callbacks=callback_list, epochs=self.epochs)

def delete_model_checkpoint(self):
    if os.path.exists(self.model_checkpoint_path):
        os.remove(self.model_checkpoint_path)

def save_model(self):
    tfjs.converters.save_keras_model(self.model, self.folder_path)
    self.delete_model_checkpoint()
```

- **__init__(self, ...):** Đây là hàm khởi tạo của lớp. Nó thiết lập các thuộc tính của lớp như kích thước ảnh (size), số epochs (epochs), kích thước batch (patch_sizes), tối ưu hóa (optimizer), hàm mất mát (loss), kích thước tập kiểm tra (test_size), số lớp phân loại (num_classes), và đường dẫn thư mục chứa dữ liệu (folder_path).
- **build_model(self):** Phương thức này xây dựng mô hình mạng nơ-ron. Nó bắt đầu bằng việc sử dụng MobileNet làm mô hình cơ sở, loại bỏ lớp đầu ra, thêm một số lớp bổ sung (GlobalAveragePooling2D, Dense, Dropout), và thiết lập lớp đầu ra tùy thuộc vào số lớp của bộ phân loại. Phương thức build_model trong lớp Python của chúng tôi được thiết kế để xây dựng một mô hình học sâu dựa trên kiến trúc MobileNet cho các nhiệm vụ phân loại hình ảnh. Dưới đây là giải thích chi tiết từng bước trong hàm này:

1. Khởi tạo MobileNet:

```
base_model = MobileNet(include_top=False, weights="imagenet",
                        input_shape=(self.size, self.size, 3))
```

Sử dụng kiến trúc MobileNet, cấu hình không bao gồm lớp đầu cuối, sử dụng trọng số đã huấn luyện từ ImageNet, và đặt kích thước đầu vào để phù hợp với thông số kích thước hình ảnh của chúng tôi.

2. Thêm Các Lớp Xử Lý:

```
x = base_model.output
x = GlobalAveragePooling2D()(x)
```

Đầu ra từ mô hình cơ sở được truyền qua lớp GlobalAveragePooling2D, giúp giảm số chiều và hỗ trợ ngăn ngừa hiện tượng quá khớp.

3. Thêm Các Lớp Dense và Dropout:

```
x = Dense(1024, activation='relu')(x)
x = Dropout(0.25)(x)
x = Dense(256, activation='relu')(x)
x = Dropout(0.2)(x)
```

Thêm các lớp Dense để học các đặc trưng cấp cao hơn, sử dụng kích hoạt 'relu' cho tính phi tuyến, và các lớp Dropout được bao gồm để giảm quá khớp bằng cách loại bỏ ngẫu nhiên một số đặc trưng.

4. Lớp Đầu Ra:

```
outs = Dense(self.num_classes, activation='softmax')(x)
```

Lớp Dense cuối cùng thiết lập số nơ-ron tương ứng với số lớp, sử dụng hàm kích hoạt 'softmax' để xuất xác suất cho từng lớp.

5. Đóng Băng Các Lớp Cơ Sở:

```
for layer in base_model.layers:
    layer.trainable = False
```

Các lớp từ mô hình cơ sở được thiết lập là không thể huấn luyện để giữ nguyên các trọng số đã huấn luyện.

6. Biên Dịch Mô Hình:

```
self.model = Model(inputs=base_model.inputs, outputs=outs)
```

Cuối cùng, mô hình được biên dịch, liên kết các đầu vào từ mô hình cơ sở với các đầu ra từ lớp Dense cuối cùng.

- **make_data(self):** Phương thức này chuẩn bị các generator dữ liệu cho quá trình huấn luyện và kiểm tra. Nó sử dụng ImageDataGenerator của Keras để thực hiện augmentation dữ liệu và tạo ra các batch ảnh và nhãn tương ứng.

1. Khởi tạo ImageDataGenerator:

```
train_datagen = ImageDataGenerator(
    preprocessing_function=keras.applications.mobilenet.preprocess_input,
    rotation_range=0.2,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.3,
    zoom_range=0.5,
    horizontal_flip=True,
    vertical_flip=True,
    validation_split=self.test_size
)
```

- `ImageDataGenerator` là công cụ sử dụng để tạo ra các phiên bản được tăng cường của hình ảnh từ dữ liệu huấn luyện ban đầu.
- Các tham số đã cấu hình quyết định loại biến đổi nào sẽ được áp dụng lên hình ảnh.
- `preprocessing_function` là hàm tiền xử lý được sử dụng để chuẩn hóa hình ảnh trước khi đưa vào mô hình.

2. Tạo `train_generator` và `validation_generator`:

```
train_generator = train_datagen.flow_from_directory(  
    self.folder_path,  
    target_size=(self.size, self.size),  
    batch_size=self.patch_sizes,  
    class_mode='categorical',  
    subset='training'  
)  
  
validation_generator = train_datagen.flow_from_directory(  
    self.folder_path,  
    target_size=(self.size, self.size),  
    batch_size=self.patch_sizes,  
    class_mode='categorical',  
    subset='validation'  
)
```

- Phương thức `flow_from_directory` tạo ra một Generator từ dữ liệu hình ảnh trong thư mục được chỉ định.
- Các tham số được truyền vào quyết định kích thước, số lượng và định dạng của dữ liệu đầu ra.

3. Trả về `train_generator` và `validation_generator`:

```
return train_generator, validation_generator
```

- Phương thức trả về hai đối tượng Generator, một cho dữ liệu huấn luyện và một cho dữ liệu kiểm thử, để sử dụng trong quá trình huấn luyện và đánh giá mô hình.

- **`train_model(self, train_generator, validation_generator)`:** Phương thức này biên dịch và huấn luyện mô hình sử dụng các generator dữ liệu huấn luyện và kiểm tra được cung cấp. Nó cũng thiết lập một callback để lưu mô hình tốt nhất dựa trên mất mát trên tập kiểm tra.
- **`delete_model_checkpoint(self)`:** Phương thức này xóa tệp kiểm tra mô hình nếu nó tồn tại. Nó được gọi sau khi lưu mô hình.
- **`save_model(self)`:** Phương thức này lưu mô hình đã huấn luyện dưới định dạng Tensor-Flow.js sau khi huấn luyện và xóa tệp kiểm tra mô hình.

5 Các Chức Năng Đã Xây Dựng

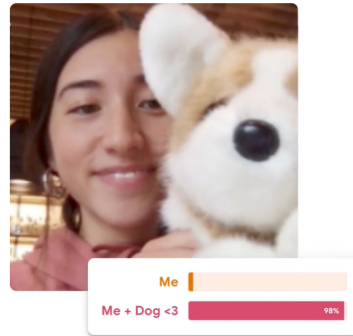
5.1 Trang chủ.

Insightful Detection

Train a computer to
recognize your own images.

A fast, easy way to create machine learning models for your
sites, apps, and more – no expertise or coding required.

Get Started

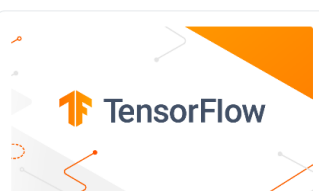


Click vào nút Get Started để chuyển trang New Project

5.2 Trang Dự Án Mới

New Project

Choose between using TensorFlow's ready-made datasets for easy training or crafting your own dataset for personalized control in machine learning training.



Standard Training

Utilize TensorFlow's pre-existing datasets to quickly train models for object detection, streamlining the process without the need for custom data gathering.



New Training

Craft your own image-based dataset for object detection, offering complete control over the data used for training models tailored to specific detection tasks.



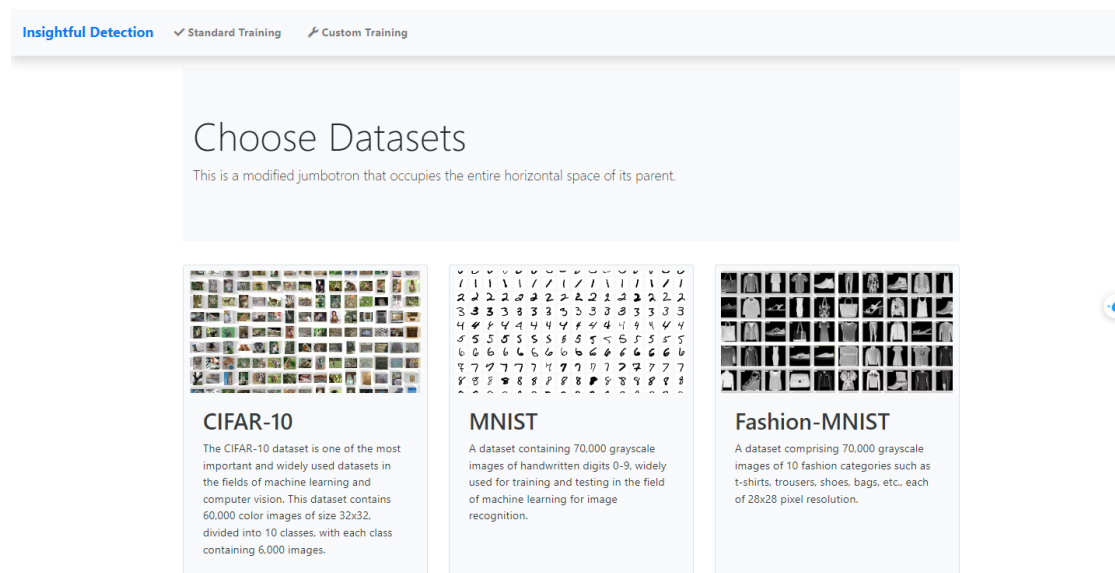
Import Training

Bringing in a pre-trained model, honed through prior training, and employing it for object detection tasks, leveraging its learned insights to make accurate predictions.

Hình 10: Trang Dự Án Mới



5.2.1 Trang Huấn Luyện Tiêu Chuẩn



Hình 11: Trang Huấn Luyện Tiêu Chuẩn



Standard Training

This dataset consists of images of five types of flowers: daisy, dandelion, roses, sunflowers, and tulips. Each image is labeled with its corresponding flower type

Training

--Select Total Images--

Train Model

daisy

10 Images Samples

Upload



dandelion

10 Images Samples

Upload



roses

10 Images Samples

Upload



sunflowers

10 Images Samples

Upload



tulips

10 Images Samples

Upload



Preview

Export Model

Choose images from your files



PREDICT

OUTPUT

daisy

dandelion

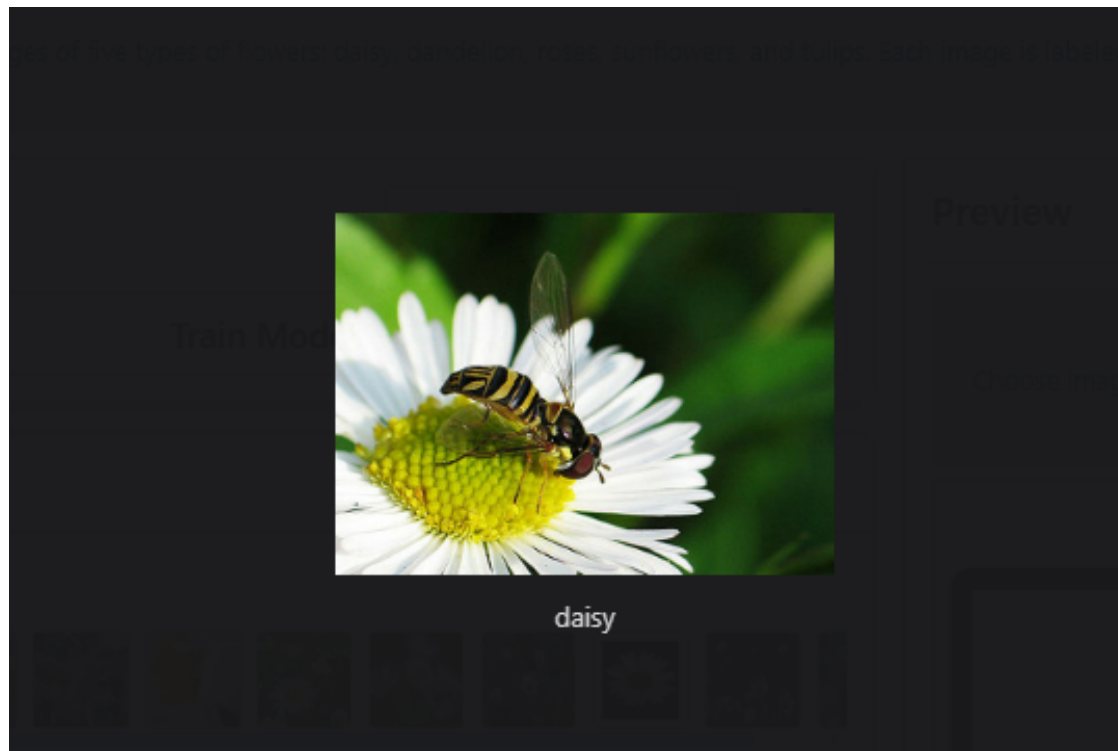
roses

sunflowers

tulips

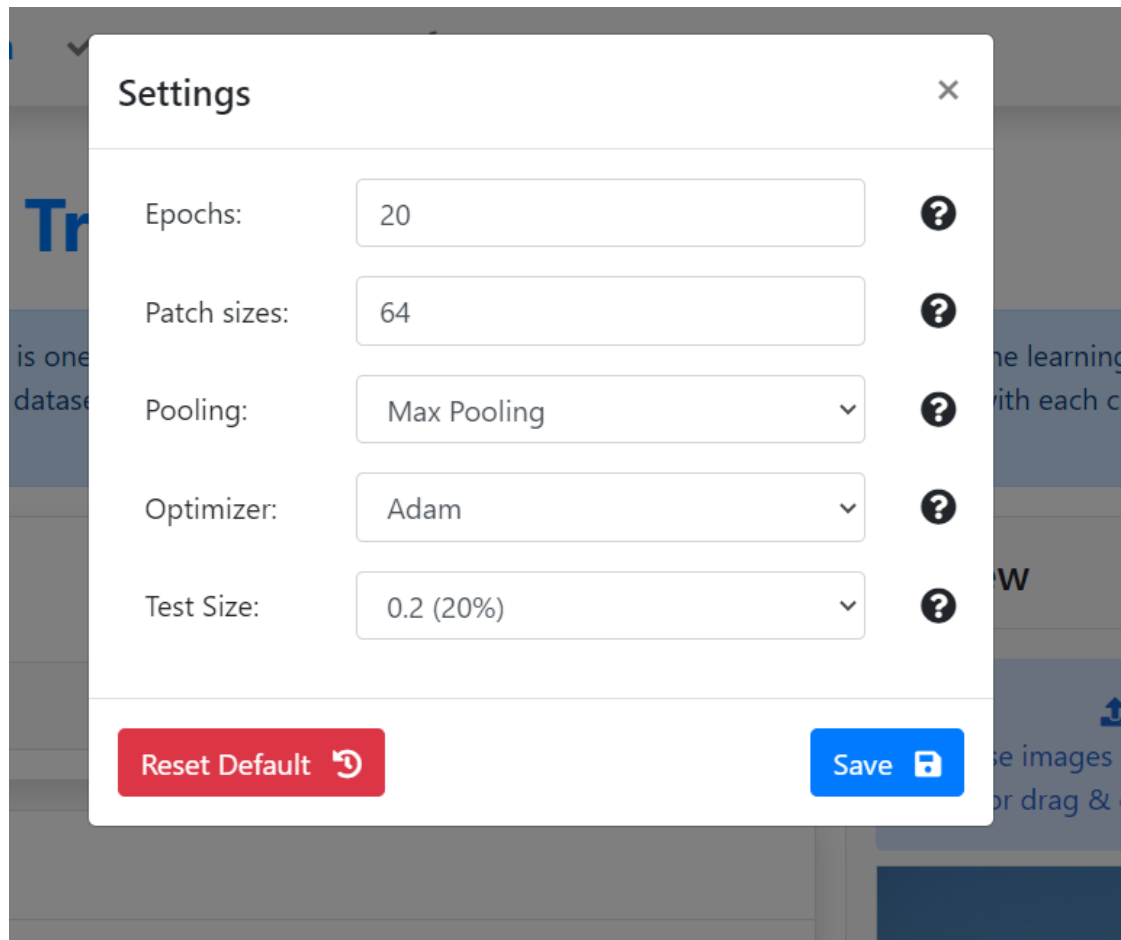
Hình 12: Chọn Bộ Dữ Liệu với Các Tùy Chọn Có Sẵn

5.2.1.1 Chọn Bộ Dữ Liệu với Các Tùy Chọn Có Sẵn



Hình 13: Xem Hình Ảnh Toàn Màn Hình

5.2.1.2 Xem Hình Ảnh Toàn Màn Hình



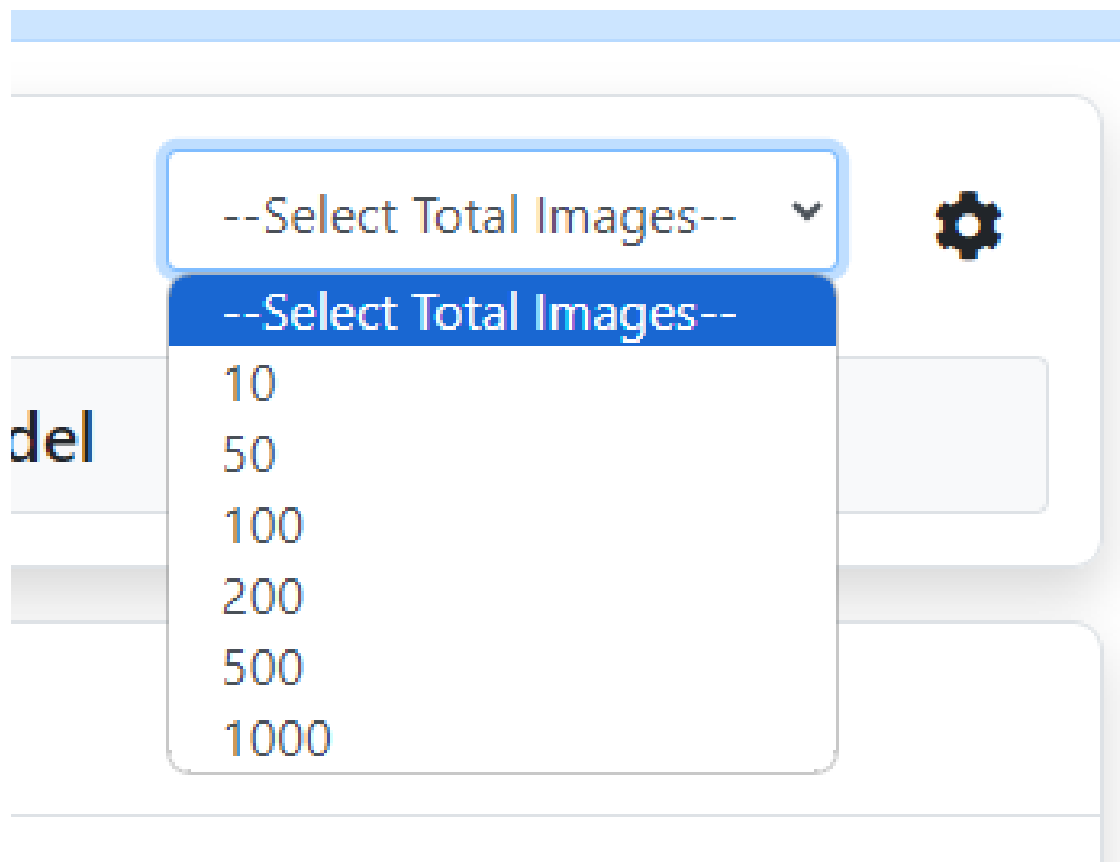
The image shows a 'Settings' dialog box with a close button (X) in the top right corner. It contains five configuration rows, each with a label, a value field, and a help icon (question mark):

- Epochs: 20
- Patch sizes: 64
- Pooling: Max Pooling (dropdown arrow)
- Optimizer: Adam (dropdown arrow)
- Test Size: 0.2 (20%) (dropdown arrow)

At the bottom of the dialog, there are two buttons: a red 'Reset Default' button with a circular arrow icon, and a blue 'Save' button with a floppy disk icon.

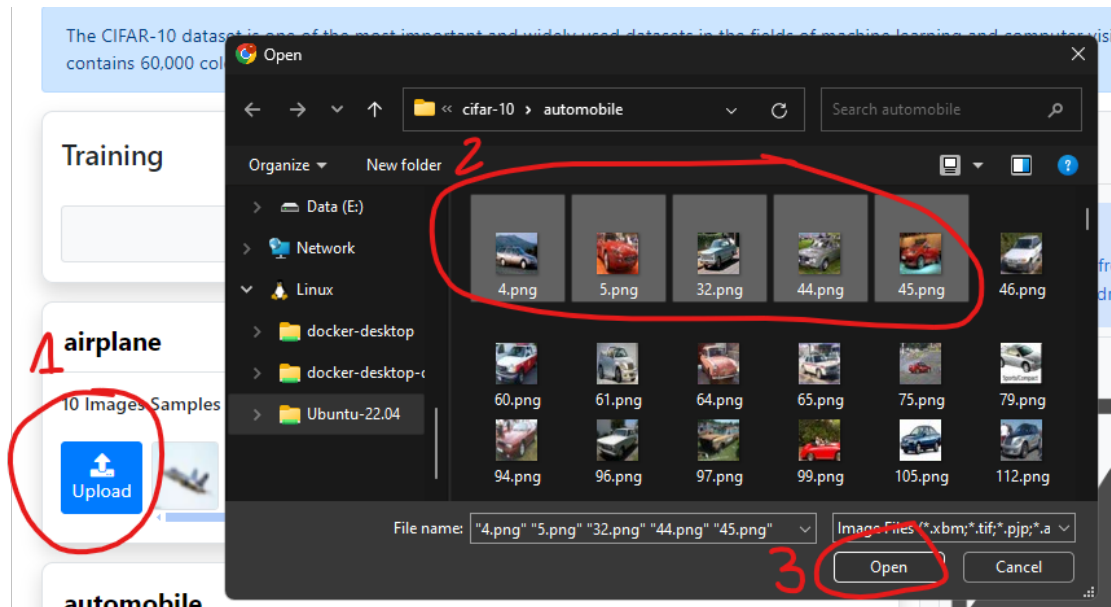
Hình 14: Biểu Mẫu Cài Đặt

5.2.1.3 Biểu Mẫu Cài Đặt

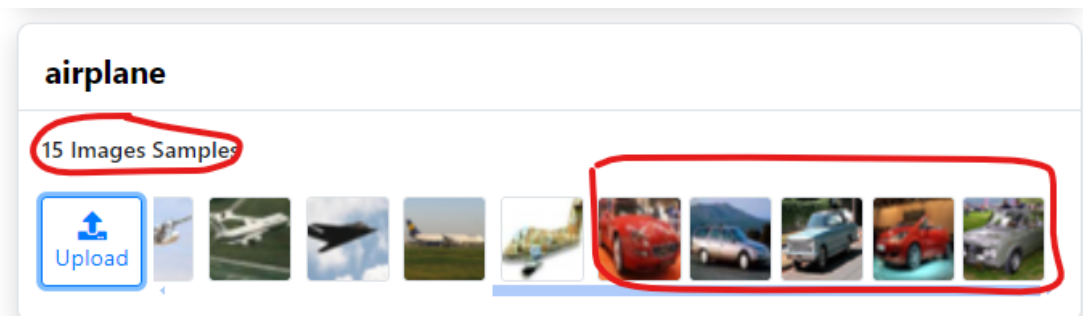


Hình 15: Chọn Tổng Số Hình Ảnh Cho Mỗi Lớp

5.2.1.4 Chọn Tổng Số Hình Ảnh Cho Mỗi Lớp



Hình 16: Chọn Hình Ảnh để Tải Lên



Hình 17: Kết Quả Chọn Hình Ảnh để Tải Lên

5.2.1.5 Chọn Hình Ảnh để Tải Lên

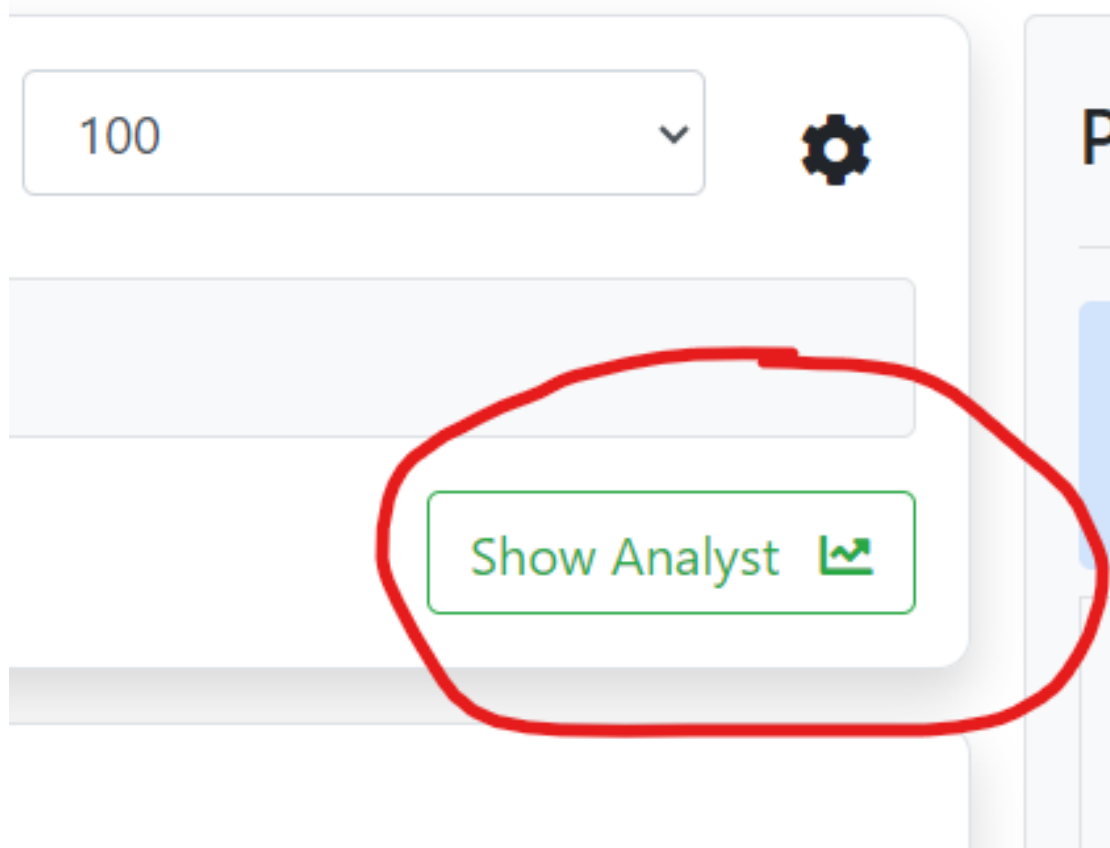
5.2.1.6 Dự Đoán và Kết Quả

5.2.2 Xử Lý Nhật Ký Huấn Luyện

```
Found 800 images belonging to 10 classes.
Found 200 images belonging to 10 classes.
Epoch 1/20
13/13 [=====] - 10s 617ms/step - loss: 3.0558 - accuracy: 0.2150 - val_loss: 1.7949 - val_accuracy: 0.3250
Epoch 2/20
13/13 [=====] - 9s 664ms/step - loss: 1.7369 - accuracy: 0.3963 - val_loss: 1.3723 - val_accuracy: 0.4850
Epoch 3/20
13/13 [=====] - 10s 788ms/step - loss: 1.4974 - accuracy: 0.4875 - val_loss: 1.2242 - val_accuracy: 0.5500
Epoch 4/20
13/13 [=====] - 8s 596ms/step - loss: 1.3698 - accuracy: 0.5163 - val_loss: 1.3264 - val_accuracy: 0.5200
Epoch 5/20
13/13 [=====] - 8s 612ms/step - loss: 1.2584 - accuracy: 0.5550 - val_loss: 1.2648 - val_accuracy: 0.5800
Epoch 6/20
13/13 [=====] - 7s 558ms/step - loss: 1.2343 - accuracy: 0.5537 - val_loss: 1.2446 - val_accuracy: 0.5100
Epoch 7/20
13/13 [=====] - 8s 579ms/step - loss: 1.2350 - accuracy: 0.5550 - val_loss: 1.2777 - val_accuracy: 0.5350
Epoch 8/20
13/13 [=====] - 9s 692ms/step - loss: 1.1712 - accuracy: 0.5813 - val_loss: 1.1930 - val_accuracy: 0.5700
Epoch 9/20
13/13 [=====] - 8s 642ms/step - loss: 1.0873 - accuracy: 0.6112 - val_loss: 1.1173 - val_accuracy: 0.5500
Epoch 10/20
13/13 [=====] - 8s 584ms/step - loss: 1.1187 - accuracy: 0.5987 - val_loss: 1.1164 - val_accuracy: 0.6300
Epoch 11/20
13/13 [=====] - 8s 645ms/step - loss: 1.0317 - accuracy: 0.6162 - val_loss: 1.1428 - val_accuracy: 0.5600
Epoch 12/20
13/13 [=====] - 8s 634ms/step - loss: 1.0306 - accuracy: 0.6538 - val_loss: 1.2075 - val_accuracy: 0.5850
Epoch 13/20
13/13 [=====] - 9s 693ms/step - loss: 1.0008 - accuracy: 0.6538 - val_loss: 1.0649 - val_accuracy: 0.6050
Epoch 14/20
13/13 [=====] - 8s 631ms/step - loss: 1.0698 - accuracy: 0.6187 - val_loss: 1.3216 - val_accuracy: 0.5750
Epoch 15/20
13/13 [=====] - 8s 620ms/step - loss: 0.9652 - accuracy: 0.6525 - val_loss: 1.0837 - val_accuracy: 0.6600
Epoch 16/20
13/13 [=====] - 8s 617ms/step - loss: 0.9060 - accuracy: 0.6725 - val_loss: 1.0658 - val_accuracy: 0.6450
Epoch 17/20
13/13 [=====] - 8s 666ms/step - loss: 1.0150 - accuracy: 0.6375 - val_loss: 1.1109 - val_accuracy: 0.5800
Epoch 18/20
13/13 [=====] - 9s 659ms/step - loss: 0.9221 - accuracy: 0.6825 - val_loss: 1.0864 - val_accuracy: 0.6050
Epoch 19/20
13/13 [=====] - 12s 965ms/step - loss: 0.9031 - accuracy: 0.6850 - val_loss: 0.9257 - val_accuracy: 0.6550
Epoch 20/20
13/13 [=====] - 12s 928ms/step - loss: 0.8351 - accuracy: 0.7050 - val_loss: 1.1879 - val_accuracy: 0.5950
```

Hình 18: Hiển thị Log ở terminal các step training ra model. Epoch chạy từ 1 đến 20 và 1-13 là số lượng tổng ảnh 800 để train / số patch_sizes là 64 làm tròn là 13. loss là hàm tính toán độ sai số, accuracy là hàm tính độ chính xác, val_loss và val_accuracy là hàm tính toán accuracy và loss trong tập data test.

5.2.3 Hiện Thị Phân Tích Mất Mát và Độ Chính Xác của Hình Ảnh



Hình 19: Hiện thị nút "Show Analysis"



Hình 20: Hiện Thị Phân Tích chỉ số Loss và Accuracy của model.



Training

--Select Total Images--

Train Model

airplane

10 Images Samples

Upload

automobile

10 Images Samples

Upload

bird

10 Images Samples

Upload

Preview

Choose images from your files, or drag & drop here

PREDICT

OUTPUT

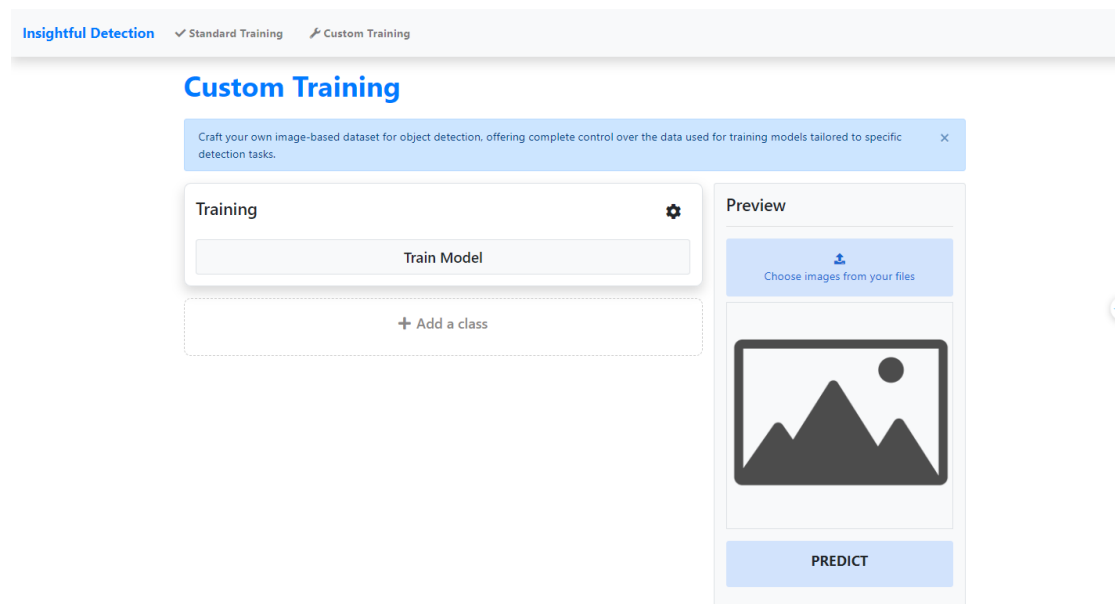
dog1

Prediction Result:
dog

Hình 21: Dự Đoán và Kết Quả



5.2.4 Trang Huấn Luyện Tùy Chỉnh



Hình 22: Trang Huấn Luyện Tùy Chỉnh

Custom Training

Craft your own image-based dataset for object detection, offering complete control over the data used for training models tailored to specific detection tasks. ×

Training

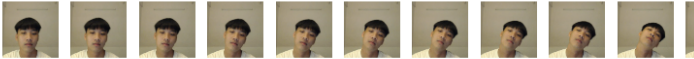


Model Trained

không đeo khẩu trang 



322 Image Samples



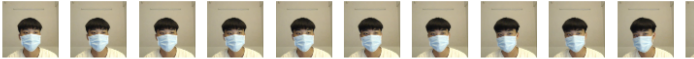
Add Image Samples:



đeo khẩu trang 



84 Image Samples



Add Image Samples:



+ Add a class

Preview

Export Model



Choose images from your files



PREDICT

Hình 23: Thêm Lớp và Thêm Hình Ảnh cho Mỗi Lớp

5.2.4.1 Thêm Lớp và Thêm Hình Ảnh cho Mỗi Lớp

Custom Training

Craft your own image-based dataset for object detection, offering complete control over the data used for training models tailored to specific detection tasks. ×

Training

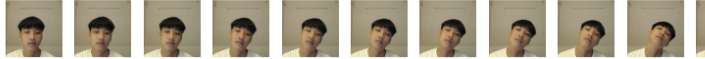


Model Trained

không đeo khẩu trang 



322 Image Samples



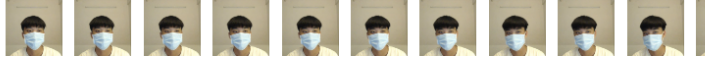
Add Image Samples:



đeo khẩu trang 



84 Image Samples



Add Image Samples:

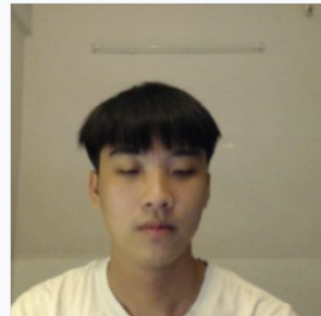


+ Add a class

Preview

Export Model


Choose images from your files



PREDICT

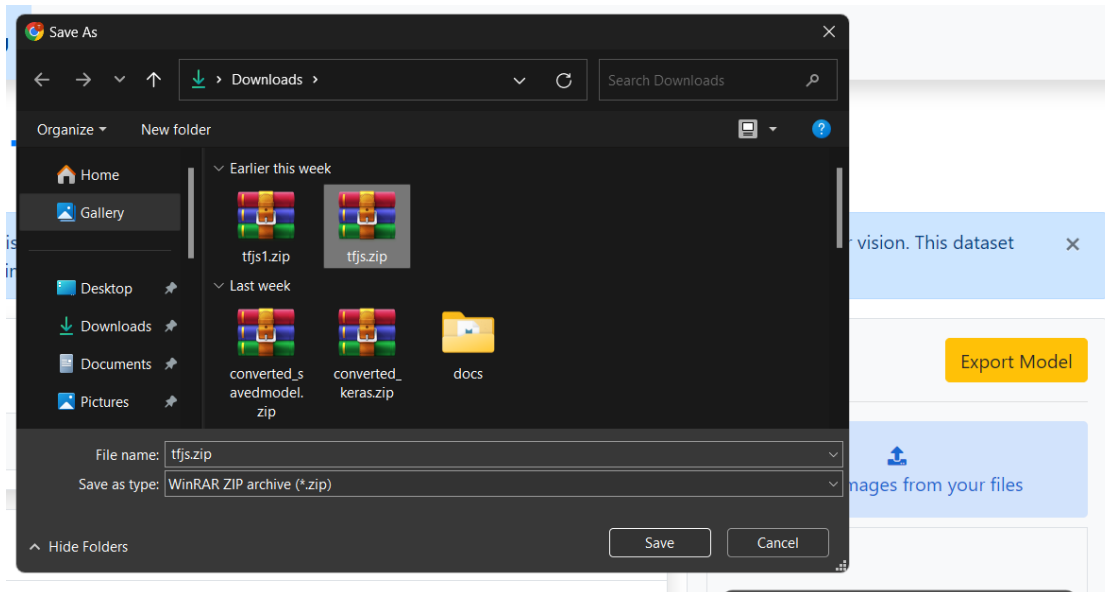
OUTPUT:

không đeo khẩu trang  1

Prediction Result:
đeo khẩu trang

Hình 24: Dự Đoán và Kết Quả

5.2.4.2 Dự Đoán và Kết Quả



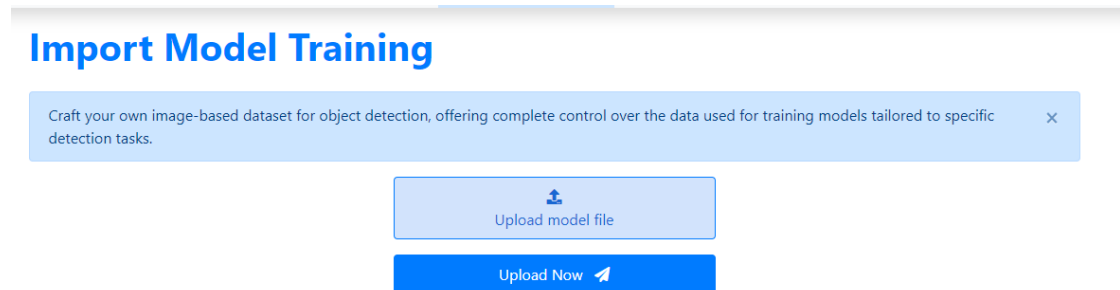
Hình 25: Xuất Mô Hình để Lưu Dưới Dạng Tệp Zip

..	
model.json	55,313
label.txt	40
group1-shard5of5.bin	1,396,520
group1-shard4of5.bin	4,194,304
group1-shard3of5.bin	4,194,304
group1-shard2of5.bin	4,194,304
group1-shard1of5.bin	4,194,304

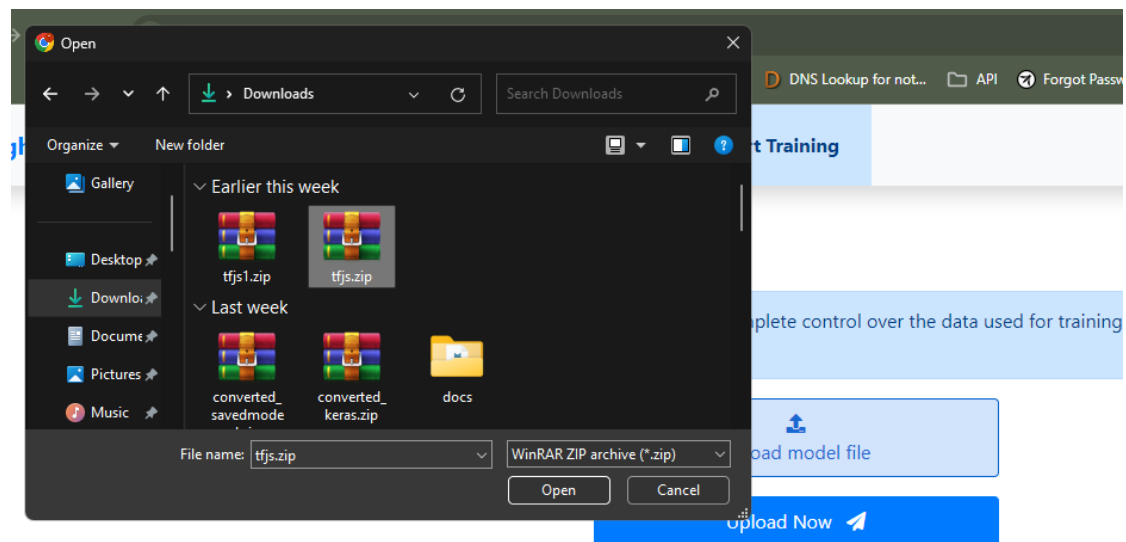
Hình 26: Xuất Dưới Dạng Zip

5.2.4.3 Xuất Mô Hình để Lưu Dưới Dạng Tệp Zip

5.2.5 Trang Nhập Mô Hình Huấn Luyện



Hình 27: Trang Nhập Mô Hình Huấn Luyện



Hình 28: Tải Lên Mô Hình Đã Huấn Luyện

5.2.5.1 Tải Lên Mô Hình Đã Huấn Luyện



Import Model Training

Choose images from your files

PREDICT

OUTPUT:

3		1
5		0
8		0

Prediction Result:
3

Hình 29: Trang Nhập Mô Hình Huấn Luyện

5.2.5.2 Trang Nhập Mô Hình Huấn Luyện



5.3 Trang Quản Trị

5.3.1 Đăng Nhập

Django administration

Username:

Password:

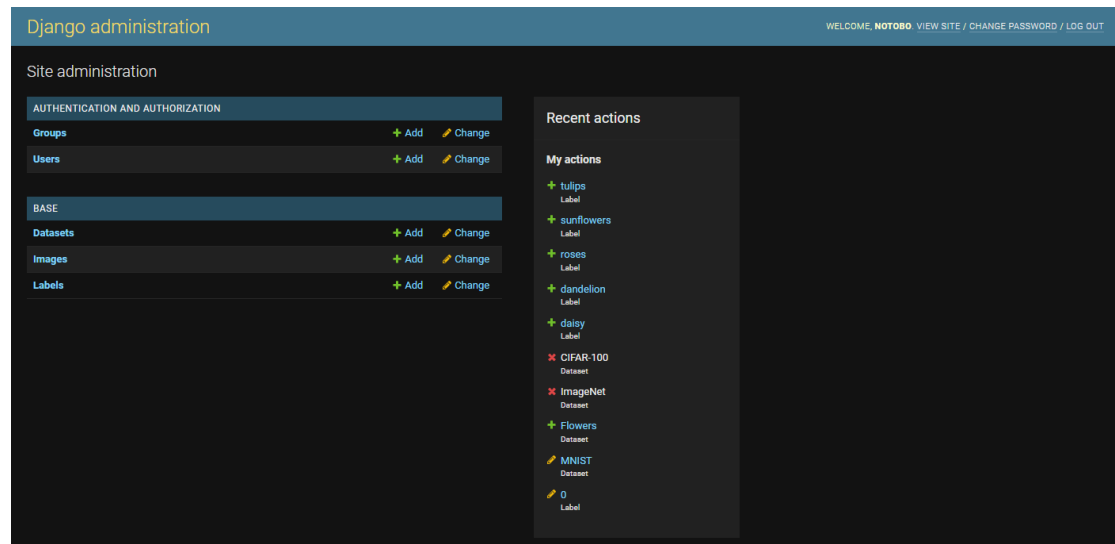
Log in

Hình 30: Đăng Nhập

Nhập tên người dùng và mật khẩu của bạn để truy cập vào tài khoản quản trị.



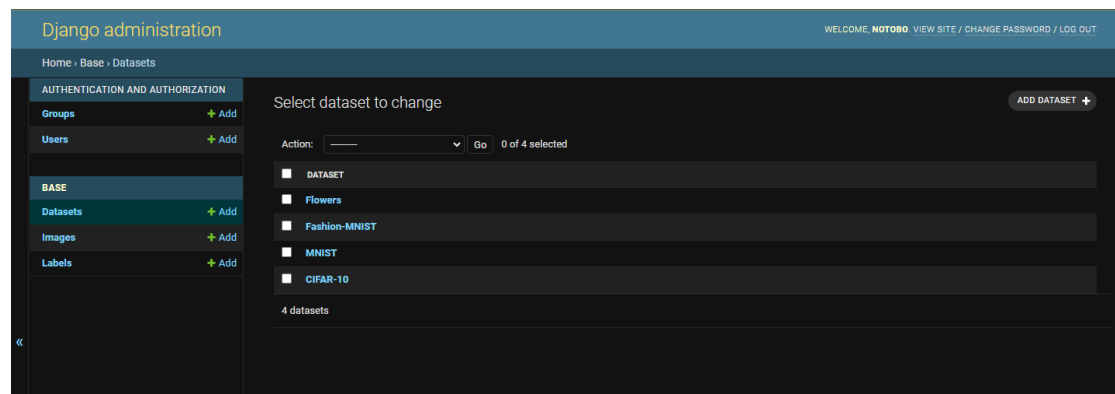
5.3.2 Bảng Điều Khiển Quản Trị



Hình 31: Bảng Điều Khiển Quản Trị

Hiển thị tất cả các mô hình trong cơ sở dữ liệu và một số mô hình mặc định.

5.3.3 Xem Thông Tin Mô Hình Bộ Dữ Liệu



Hình 32: Xem Thông Tin Mô Hình Bộ Dữ Liệu



5.3.4 Tạo Mô Hình Bộ Dữ Liệu Mới

The screenshot shows the Django administration interface for adding a new dataset. The left sidebar contains a navigation menu with sections 'AUTHENTICATION AND AUTHORIZATION' (Groups, Users) and 'BASE' (Datasets, Images, Labels). The 'Datasets' section is selected. The main content area is titled 'Add dataset' and contains four form fields: 'Name', 'Slug', 'Description' (a large text area), and 'Img'. At the bottom right of the form are three buttons: 'Save and add another', 'Save and continue editing', and 'SAVE'.

Hình 33: Tạo Mô Hình Bộ Dữ Liệu Mới

5.3.5 Xóa Mô Hình Bộ Dữ Liệu

The screenshot shows a confirmation dialog box with the title 'Are you sure?'. The main text asks: 'Are you sure you want to delete the dataset "CIFAR-10"? All of the following related items will be deleted:'. Below this, there are two sections: 'Summary' with a bullet point 'Datasets: 1' and 'Objects' with a bullet point 'Dataset: CIFAR-10'. At the bottom, there are two buttons: 'Yes, I'm sure' (in red) and 'No, take me back' (in grey).

Hình 34: Xóa Mô Hình Bộ Dữ Liệu



5.3.6 Chỉnh Sửa Mô Hình Bộ Dữ Liệu

The screenshot displays the Django administration interface for editing a dataset. The left sidebar shows the navigation menu with 'BASE' > 'Datasets' selected. The main content area is titled 'Change dataset' and shows the details for the 'Fashion-MNIST' dataset. The form includes fields for 'Name' (Fashion-MNIST), 'Slug' (fashion-mnist), 'Description' (A dataset comprising 70,000 grayscale images of 10 fashion categories such as t-shirts, trousers, shoes, bags, etc., each of 28x28 pixel resolution), and 'Img' (Fashion-MNIST.webp). At the bottom, there are buttons for 'Delete', 'Save and add another', 'Save and continue editing', and 'SAVE'.

Hình 35: Chỉnh Sửa Mô Hình Bộ Dữ Liệu



6 Nhiệm vụ, vai trò của từng thành viên trong nhóm.

STT	Họ Tên	Công Việc
1	Nguyễn Thanh Bình	Xây dựng CNN, views, model, cơ sở dữ liệu, viết báo cáo

Bảng 1: Bảng phân chia công việc



Tài liệu

- [1] Keras Documentation, <https://keras.io/>
- [2] OpenCV Documentation, https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
- [3] Django Documentation, *Official Django Documentation*, Version 5.0, <https://docs.djangoproject.com/en/5.0/>
- [4] TensorFlow Documentation, *Official TensorFlow Documentation*, https://www.tensorflow.org/api_docs
- [5] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, Hartwig Adam, *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*, arXiv preprint arXiv:1704.04861, April 2017, <https://arxiv.org/abs/1704.04861>
- [6] Rikiya Yamashita, Mizuho Nishio, Richard Kinh Gian Do, & Kaori Togashi. (2018). *Convolutional neural networks: an overview and application in radiology*. Radiological Physics and Technology, 12(3), 235-248.
- [7] Vũ Hữu Tiệp, *Deep Learning*, Machine Learning Cơ Bản, July 6, 2018, <https://machinelearningcoban.com/2018/07/06/deeplearning/>.